

Präsentationsleitfaden für Spendly

Darian Germann und Alex Platret | 1. Oktober 2026

Dieser Leitfaden ergänzt die Folien mit einem Ablauf für die Live-Demo und Antworten auf technische Rückfragen. Die vorgeschlagenen Zeiten sind eine Probeplanung von rund neun Minuten; eine Vorgabe des Dozenten geht vor.

Die Aussage der Präsentation

Spendly zeigt, wie tägliche Ausgaben das verfügbare Monatsbudget verändern. Die App speichert lokal und verbindet einen kurzen Eingabeweg mit verständlichen Rückmeldungen und einer Gerätesperre.

Abschnitt	Zeit
Aufgabe, Zielgruppe und Vergleich mit dem Entwurf	1:55
Live-Demo vom Entsperren bis Rückgängig	2:00
Biometrie und technische Realisierung	1:50
Konzeptvergleich, Qualität und Herausforderungen	2:25
Rückblick und nächster Schritt	0:45

Demo in zwei Minuten

1. Spendly entsperren. Kurz sagen, dass der Scan vom Betriebssystem geprüft wird und der eigene App-Code als Alternative bleibt.
2. Das aktuelle Monatsbudget zeigen. Restbudget und Tagesorientierung in einem Satz erklären.
3. Eine Ausgabe mit CHF 14.80, Essen und Trinken sowie «Mittagessen» erfassen. Den veränderten Budgetstand zeigen.
4. Im Verlauf nach «Mittagessen» suchen und den Betrag auf CHF 12.80 korrigieren.
5. Den Eintrag löschen und innerhalb von fünf Sekunden «Rückgängig» wählen. Die App schliessen, neu öffnen, entsperren und den erhaltenen Eintrag zeigen.

Vor dem Vortrag

Den vorgesehenen Build öffnen, Sperrcode kennen, Monatsbudget und Beispieldaten kontrollieren. Gerät laden, Bildschirm Spiegelung testen und persönliche Mitteilungen ausblenden. Die Demo einmal vollständig proben. Screenshots oder die Sicherungsversion der Folien bereithalten, falls die Verbindung ausfällt.

Die Präsentation verwendet Inter und Newsreader. Falls die PPTX anders aussieht, die mitgelieferten Schriften installieren oder die PDF mit festem Layout verwenden. Der Sprechertext steht zusätzlich als eigene Datei bereit.

Den Code nachvollziehbar erklären

Eine einzige Ausgabe eignet sich als roter Faden durch den Code. Zuerst das sichtbare Verhalten zeigen und erst danach die beteiligten Dateien öffnen.

Station	Datei	Erklärung in eigenen Worten
Eingabe	screens/NewExpenseScreen.js	State hält die Eingaben. Die Prüfung bestimmt, ob Speichern möglich ist.
Regeln	utils/validation.js	Jede Regel liefert entweder keinen Fehler oder einen verständlichen Text.
Koordination	storage/DataContext.js	Eine Aktion schreibt Daten und aktualisiert danach die Ansichten.
Speicherung	storage/expenses.js	SQL arbeitet mit gebundenen Parametern und Beträgen in Rappen.
Budget	utils/budget.js	Restbudget und verbleibende Tage ergeben die Tagesorientierung.
Sperre	utils/biometrics.js und storage/pin.js	Systemprüfung und eigener Code sind getrennte Wege zum Entsperren.
Navigation	navigation/RootNavigator.js	Stack, Tabs und Modal bilden den Ablauf; die Sperre setzt ihn zurück.

Drei kleine Beispiele

Rappen: Aus CHF 14.80 wird 1480. Die Summe bleibt eine Ganzzahl. Erst die Anzeige formatiert zurück in CHF.

Budget: Am 23. September bleiben bei CHF 800.00 Budget und CHF 604.00 Ausgaben noch CHF 196.00. Acht Tage einschliesslich heute ergeben CHF 24.50 pro Tag.

Fehlerfall: Ein Betrag von null ist ungültig. Das Feld zeigt den Grund, und der Speichern-Button bleibt gesperrt. Schlägt das Schreiben fehl, müssen Eingaben erhalten bleiben.

Wichtige Begriffe

State ist der aktuelle Zustand einer Ansicht. Context stellt gemeinsame Daten mehreren Screens bereit. Eine Promise steht für ein noch laufendes Ergebnis; await wartet darauf. SQL speichert und liest strukturierte Daten. Eine Migration passt eine bestehende Datenbank an eine neue Version an.

Im Vortrag nur Code zeigen, der erklärt werden kann. Auf Rückfragen erst den Zweck nennen, danach die konkrete Funktion. Keine Zeile nur vorlesen.

Antworten auf mögliche Rückfragen

Die Antworten beschreiben den tatsächlichen Umfang. Bei einem nicht durchgeführten Test wird das offen gesagt.

Warum React Native und Expo

Eine gemeinsame JavaScript-Basis kann native iOS- und Android-Oberflächen verwenden. Expo stellt die benötigten Gerätemodule bereit. Native Funktionen müssen trotzdem auf dem passenden Build und Gerät geprüft werden.

Warum SQLite

Die App benötigt strukturierte lokale Daten, einzelne Änderungen und Summen je Monat. SQL passt zu diesen Beziehungen. Ein Server ist für den gewählten Umfang nicht nötig.

Sind die Daten verschlüsselt

Die SQLite-Datenbank ist nicht verschlüsselt. Die App-Sperre schützt den Zugang in Spendly. Der Code-Prüfwert liegt separat in SecureStore; Gesicht und Fingerabdruck erhält Spendly nicht.

Warum ist SHA256 noch eine Grenze

Ein Salt verhindert gleiche Prüfwerte bei gleichen Codes. Ein sechsstelliger Code hat trotzdem einen kleinen Suchraum. Eine aufwendigere Passwortableitung wäre stärker; die aktuelle Wartezeit begrenzt Versuche in der App.

Was wurde gegenüber LB1 geändert

Monatsbudgets, dauerhafte Löschrufen und gespeicherte Code-Fehlversuche wurden vervollständigt. Dark Mode kam hinzu. Die stärkere Passwortableitung und der gemessene Zehn-Sekunden-Nutzertest bleiben offen.

Was beweisen die Tests

91 automatisierte Tests in neun Testdateien bestehen. Davon führen neun SQLite-Tests echtes SQL in einer Speicherdatenbank aus. Zwei Tests prüfen den Wiederherstellungsdialog mit aktivierter und deaktivierter Biometrie. Ein tatsächlicher Scan braucht zusätzlich einen Gerätetest.

Was wäre der nächste Schritt

Zuerst Zugangsschutz weiter verbessern und die Bedienung auf zusätzlichen Geräten mit Menschen testen. Die Erfassungszeit und die Verständlichkeit von Fehlern sind messbare Ziele.

Persönlicher Rückblick: Vor dem Vortrag ein eigenes konkretes Beispiel ergänzen, welche Entscheidung ihr begründet habt und welche Stelle ihr heute anders angehen würdet. Keine nicht erlebte Schwierigkeit behaupten.